

Strategic Shift for ORDINA after Identifying DISBIOME

Rafael Correia

May 11, 2026

Abstract

ORDINA began as a Django-based curated database and browser for microbiome–disease evidence, with support for study records, study groups, comparisons, qualitative taxon findings, quantitative taxon values, lineage-aware taxonomy, preview-first imports, and exploratory graph views. The discovery of DISBIOME changes the strategic context of the project. DISBIOME already occupies much of the space of a manually curated microbiome–disease association database. Therefore, ORDINA should not be positioned as a competing static database. Instead, ORDINA should shift toward a provenance-first evidence integration and network-analysis platform: a system that can ingest external databases such as DISBIOME, semi-automatically triage and extract new literature evidence, preserve human review as a quality-control step, and expose signed, lineage-aware disease–organism networks with conflict and support summaries. This shift should include two deliberate rewrites. First, the current import subsystem should be treated as a useful prototype, but not as the foundation for the next version. Imports, source synchronization, graph precomputation, and pipeline-derived evidence generation should move into API-triggered Docker workers coordinated by Celery and Redis, with PostgreSQL remaining the canonical relational store. Second, the current network layer should be treated as a prototype and semantically redesigned so that biological entities are not duplicated by visual state. For example, enriched and depleted evidence should not create two copies of the same organism node. ORDINA Retes should instead use canonical disease and organism nodes with directed, signed, evidence-backed edges and explicit support/conflict summaries.

1 Current State of ORDINA

The current ORDINA repository already contains a useful foundation rather than a disposable prototype. Its README defines ORDINA as the *Organism-Resolved Disease Interaction Network Atlas*, a Django-based database and browser for curated microbiome study evidence. The current data model is centered on study-level records, groups or cohorts, explicit comparisons, lineage-aware taxon records, qualitative findings, quantitative findings, flexible metadata, preview-first import workflows, and disease/co-abundance graph exploration [1].

The implemented schema is compact and analysis-oriented. The main models include `Study`, `Group`, `Comparison`, `Taxon`, `TaxonClosure`, `TaxonName`, `QualitativeFinding`, `QuantitativeFinding`, `MetadataVariable`, `MetadataValue`, and `ImportBatch`. This is a good basis for reproducible curation because it distinguishes paper-level evidence, group-level context, comparison-level directionality, taxonomic normalization, and import provenance [2].

The graph layer is also already aligned with the new direction. The current project exposes two server-rendered graph views derived from curated `QualitativeFinding` rows: a disease network and a co-abundance taxon network. These graphs are generated from relational evidence at request time, with row-level traceability back into the browser. This is important because ORDINA’s strongest differentiator should be evidence-backed network exploration, not simply storing association rows [4].

The current import pipeline is admin-only and preview-first. Uploaded CSV or Excel files are parsed and validated before any database write, and confirmed imports create `ImportBatch`

records. Conceptually, this preview-first idea is valuable: validate first, show the curator what will happen, then write with provenance [3]. However, the implementation should not be treated as the long-term import architecture. The next version of ORDINA should keep the lessons and the data-contract discipline, but scrap the current import subsystem as an implementation unit and replace it with an asynchronous, API-driven worker architecture.

2 Why the Project Needs to Shift

DISBIOME changes the competitive landscape. DISBIOME is a database that collects published microbiota–disease information in a standardized way. Its disease names are classified using MedDRA, microorganisms are linked to NCBI and SILVA taxonomy, and included studies are assessed for reporting quality using a standardized questionnaire [8]. This overlaps strongly with a simple interpretation of ORDINA as a manually curated microbiome–disease association database.

This means that ORDINA should avoid the weak claim:

ORDINA is a new curated database of microbiome–disease associations.

That claim is too close to DISBIOME. A stronger claim is:

ORDINA is a provenance-first evidence integration and network-analysis platform for microbiome–disease associations.

The important problem is not that manual curation is useless. The problem is that manual curation alone does not scale. MiDRED frames this directly: domain experts manually review publications to extract microbe–disease associations, but manual curation struggles to keep up with publication volume. MiDRED was created as a human-annotated dataset of fine-grained microbe–disease relations to support text-mining solutions for microbiome knowledge-base construction [9].

Therefore, ORDINA should not try to beat DISBIOME by manually curating more papers. ORDINA should instead make manual review cheaper, more traceable, and more analytically useful.

3 Proposed New Positioning

The strategic shift should be:

From: a manually curated microbiome–disease database.

To: a provenance-first system that integrates external databases, semi-automated literature mining, manual review, and reproducible analysis into disease–microbe evidence networks.

A concise project description could be:

ORDINA is an organism-resolved disease interaction network atlas that converts microbiome association evidence from curated databases, literature, and reproducible pipelines into reviewable evidence statements and analyzable disease–organism networks.

This makes DISBIOME an input source and validation of the field, not a project killer. ORDINA’s value becomes the integration layer, the review workflow, the evidence graph, and the ability to compare and explain conflicting findings.

4 Recommended Module Structure

The project can be described as four related components. These do not need to become separate repositories immediately. At first they can be logical modules inside the same Django project.

Module	Purpose	Role in the shift
ORDINA Core	Canonical database, schema, provenance, import audit, and normalized entities.	Keeps the existing Django/PostgreSQL foundation but adds explicit source records, evidence statements, review states, and external identifiers.
ORDINA Retes	Network and graph layer.	Evolves the existing disease and co-abundance graphs into evidence-weighted, conflict-aware, lineage-aware networks.
ORDINA Flux	Import, source synchronization, and pipeline layer.	Replaces the current import subsystem with API-triggered Celery/Redis worker jobs for DISBIOME import, database adapters, candidate evidence generation, graph precomputation, and later raw-read or abundance-table pipelines.
ORDINA Curia	Human-in-the-loop review layer.	Turns manual curation into review, correction, approval, and audit of machine/import-generated candidate evidence.

The names can be adjusted later, but the conceptual split is useful. ORDINA Core stores normalized evidence. ORDINA Flux brings evidence in. ORDINA Curia reviews it. ORDINA Retes turns it into explorable networks.

5 Core Data Model Shift

The current schema should not be thrown away. The correct move is to add a provenance and candidate-evidence layer around the existing study, group, comparison, taxon, and finding models.

5.1 Current Model

The current conceptual model is:

Study

- > Group
- > Comparison
- > QualitativeFinding
- > QuantitativeFinding
- > Taxon / TaxonClosure / TaxonName
- > ImportBatch

This is good for curated workbook imports. However, it assumes the main input is a curator-prepared CSV or Excel contract.

5.2 Proposed Additional Layer

ORDINA should add a source-first layer:

Source

- > SourceDataset
- > SourceRecord
- > ImportRun
- > CandidateEvidenceStatement
- > ReviewDecision
- > EvidenceStatement
- > GraphEdgeSummary

The important idea is that not every imported or extracted association immediately becomes trusted curated evidence. Data should pass through states:

raw_source_record

- > candidate_evidence
- > normalized_candidate
- > human_reviewed_evidence
- > graph_edge_summary

A possible set of review states is:

imported_unreviewed
machine_suggested
needs_review
approved
edited_and_approved
rejected
superseded

This allows ORDINA to represent both high-confidence manual evidence and lower-confidence imported candidates without mixing them silently.

6 Import Architecture Shift: Replace the Current Import System

The import layer should be the first part of ORDINA to be rewritten. This is not because the current import concept is wrong, but because imports are about to become more complex than manually uploading a prepared workbook. ORDINA now needs to support external database synchronization, source staging, candidate generation, taxon resolution, review queues, graph summary updates, and eventually pipeline-derived evidence. These jobs should not run inside normal Django web requests or remain tightly coupled to an admin upload form.

The recommendation is therefore:

Scrap the current import subsystem as an implementation, but preserve the useful ideas: explicit contracts, preview-first validation, provenance, and auditability.

The replacement should be an API-triggered asynchronous import system built around Dockerized workers. The Django application should expose endpoints or service functions that create import jobs, validate permissions, record job metadata, and enqueue work. Dedicated

workers should then execute the heavy work outside the request cycle. Celery is appropriate for this role because it is designed to distribute work across threads or machines using task queues, worker processes, and a broker between clients and workers [11]. Redis is a reasonable broker for the MVP because Celery documents Redis broker and result-backend support directly [12].

A practical architecture is:

Django web/API

- > creates ImportRun / Job row in PostgreSQL
- > enqueues Celery task through Redis
- > returns job identifier to UI

Celery worker container

- > fetches source data or uploaded artifact
- > writes raw SourceRecord rows
- > normalizes into CandidateEvidenceStatement rows
- > performs resolver checks and validation
- > updates job progress/status

Curia review UI

- > displays candidate evidence
- > approves, edits, rejects, or requests resolver review

Retes graph tasks

- > recompute affected graph summaries asynchronously
- > store cached GraphEdgeSummary / graph artifact rows

PostgreSQL should remain the canonical store for relational state: studies, groups, comparisons, taxa, source records, candidate evidence, review decisions, import jobs, and graph summaries. Redis should not be treated as the database of record. It should be used as a broker, short-lived cache, and task coordination layer. In other words, the goal is not to “scale to PostgreSQL”; ORDINA already uses PostgreSQL. The goal is to scale the work *around* PostgreSQL by separating web requests from long-running imports and by allowing more worker containers to be added when import or graph-precomputation load increases.

This architecture also makes the project easier to organize. Instead of one large import path, ORDINA Flux can contain separate task families:

- `source.sync`: fetch or refresh external source records, such as DISBIOME;
- `source.stage`: persist raw source payloads without interpreting them as truth;
- `evidence.normalize`: map source records into candidate evidence statements;
- `taxonomy.resolve`: resolve organism names and mark uncertain taxa for review;
- `curia.prepare`: prepare review queue entries;
- `retes.summarize`: recompute support, conflict, and graph-edge summaries;
- `pipeline.run`: later run raw-read or abundance-table workflows;
- `export.build`: generate downloadable datasets or graph exports.

The main risk is over-engineering. To avoid this, the first worker implementation should be small: one Redis service, one Celery worker service, one Django web service, and PostgreSQL. The first job type should be the DISBIOME import/staging job. More queues and worker classes should only be added once there is a measured need.

7 DISBIOME as the First Source Adapter

The first practical milestone should be a DISBIOME adapter. DISBIOME should be treated as a source feed, not as something to copy blindly into ORDINA’s canonical tables.

7.1 Recommended Flow

```
DISBIOME export / API
-> raw_disbiome_source_record
-> mapping preview
-> normalized ORDINA candidate evidence
-> curator review / import approval
-> EvidenceStatement and graph edges
```

This keeps provenance explicit. Each imported association should retain at least:

- source database name;
- source record identifier;
- source URL when available;
- source publication identifier;
- organism name as written;
- disease name as written;
- normalized taxon identifier;
- normalized disease identifier when available;
- evidence direction;
- method, sample type, country, and other metadata where available;
- review state;
- import run identifier.

The advantage is that ORDINA can use DISBIOME to seed a useful graph quickly, while still adding value through normalization, network summarization, conflict detection, and review.

8 Fixing the Manual Curation Problem

The manual curation problem should be reframed. The goal is not to eliminate human curation. The goal is to move humans from full manual entry to targeted review.

8.1 Literature Triage

ORDINA Flux should eventually watch PubMed, Europe PMC, or other literature sources for new microbiome–disease studies. The output should not be automatic database entries at first. It should be a ranked queue of candidate papers.

A simple first version can rank papers using keywords, entity co-mentions, publication year, human host terms, disease terms, microbiome terms, and availability of full text or supplementary tables. A better later version can use entity and relation annotations from systems such as PubTator 3.0, which provides semantic and relation search over biomedical literature and offers API/bulk access to annotations [10].

8.2 Entity Extraction

The next step is candidate entity extraction:

paper

- > candidate diseases
- > candidate organisms
- > candidate sample types
- > candidate methods
- > candidate evidence sentences

Taxon normalization should remain strict. If an organism cannot be resolved confidently, ORDINA should mark it as review-required rather than silently creating a weak taxon. This matches the current preview-first behavior where review-required taxa remain visible but dependent findings can be skipped or blocked from confirmed import.

8.3 Relation Extraction

Relation extraction is the hard part. ORDINA needs to distinguish:

- organism increased in disease;
- organism decreased in disease;
- organism merely mentioned near a disease;
- organism discussed as background;
- no significant change;
- pathogen or causal language;
- conflicting statements across studies.

MiDRED is relevant here because it directly targets microbe–disease relation extraction and was designed to support automated microbiome knowledge-base construction [9]. ORDINA should not begin by training a large model. A reasonable first approach is:

rules + entity annotations + evidence sentence extraction + review UI

Only later, once ORDINA has enough reviewed corrections, should it consider model training or fine-tuning.

8.4 Human-in-the-loop Review

ORDINA Curia should become a central feature. A reviewer should see a compact proposed statement:

Paper: <title / DOI / PMID>
Sentence or table row: <evidence text>
Disease: <candidate disease>
Organism: <candidate organism>
Taxon resolution: <candidate NCBI/SILVA/local mapping>
Direction: increased / decreased / no relation / unclear
Sample type: stool / saliva / tissue / unknown
Method: 16S / shotgun / qPCR / culture / unknown
Confidence: <score>
Source: DISBIOME / PubTator / manual / pipeline

The reviewer actions should be:

Approve
Edit and approve
Reject

Mark as needs second review
Mark organism resolution problem
Mark disease normalization problem

This changes manual curation from spreadsheet filling into quality control.

9 Network Shift: From Display to Evidence Analysis

ORDINA Retes should build on the existing graph pages. The repo already supports a disease graph and a co-abundance graph, with Cytoscape/ECharts renderers, taxonomic rollup, filters, support tables, and graph exports [5]. The next shift is to make graph edges evidence-aware.

9.1 Current Graph Semantics

The current disease graph is comparison-centered. Disease nodes are derived from the target group condition or name; enriched/increased taxa and depleted/decreased taxa are rendered on separate sides; edges aggregate findings after taxonomic rollup; and browser links provide access back to supporting evidence [6].

The current co-abundance graph summarizes repeated taxon-pair patterns across shared qualitative comparison contexts, classifying support as same-direction, opposite-direction, or mixed [7].

9.2 Proposed Next Graph Semantics

The next step is not only to draw more graphs. It is to calculate graph edge meaning from evidence summaries:

```
Disease -- organism edge:
  support_count
  study_count
  source_count
  increased_count
  decreased_count
  no_change_count
  conflict_score
  review_tier
  source_breakdown
  method_breakdown
  sample_type_breakdown
```

This would allow ORDINA to show edges such as:

```
Crohn disease -- Faecalibacterium
  mostly decreased
  high support
  low conflict
  mostly stool studies
  mixed sequencing methods
```

or:


```
Disease X -- Taxon Y
  conflicting evidence
  5 increased, 4 decreased
  method/sample dependent
  needs review
```

This is more useful than a binary association graph and more honest than hiding conflicting findings.

10 Network Architecture Rewrite: Canonical Nodes, Signed Edges, and Better Semantics

The current network layer should also be redesigned. This is a different issue from the import rewrite. The import rewrite is about execution and scalability. The network rewrite is about semantics: what nodes and edges actually mean.

The most important correction is that ORDINA should not create duplicate organism nodes depending on whether an organism is enriched or depleted. This makes the visual graph easier to draw in the short term, but it breaks the biological meaning of the network. The same taxon is the same entity even when evidence about it differs across diseases, studies, sources, methods, or directions. Enrichment and depletion are properties of evidence statements or summarized edges, not separate organism identities.

A better disease–organism network should use canonical entity nodes:

```
Disease node: Crohn disease
Taxon node: Faecalibacterium prausnitzii
Evidence edge(s): disease -> taxon
  sign = decreased_in_target
  support_count = ...
  source_count = ...
  review_tier = ...
  conflict_score = ...
```

In this model, a disease and an organism appear once in a given graph scope. The relationship between them carries the biological and evidential meaning. If there are both increased and decreased findings for the same disease–organism pair, ORDINA should not create two organism nodes. It should show one disease–organism relationship with a conflict summary:

```
Disease X -> Taxon Y
  increased_count = 5
  decreased_count = 4
  dominant_direction = mixed
  conflict_score = high
  evidence_status = needs_review_or_context_split
```

The edge direction should also be defined carefully. In the simplest disease–organism view, the arrow can mean “evidence is stated from the target condition/comparison toward the organism”, while the enrichment/depletion state is stored as an edge sign or relation type. This avoids pretending that the graph direction necessarily means causality. Later, ORDINA can add separate causal or mechanistic relations only when the evidence actually supports them.

The recommended distinction is:

- **Evidence-level graph:** a detailed graph or table where each `EvidenceStatement` is preserved separately. Multiple studies can create multiple edges between the same disease and organism, so this layer may behave like a directed multigraph.
- **Summary graph:** an aggregated graph where each disease–taxon pair appears once per graph scope, with support counts, direction counts, source breakdown, review status, and conflict metrics.
- **Projection graphs:** derived graphs such as disease–disease similarity or organism–organism co-patterns. These should be computed from the evidence matrix and clearly labelled as projections, not treated as directly observed biological interactions.

This also affects the co-abundance graph. Repeated same-direction or opposite-direction taxon pairs across disease comparisons can be useful, but they should not automatically be interpreted as ecological co-abundance or direct taxon–taxon interaction. ORDINA should label these as evidence-derived co-patterns unless they come from actual abundance correlation or co-occurrence analyses.

10.1 Research Note: Best Improvements for ORDINA Retes

Before deeply rewriting ORDINA Retes, the project should include a small research phase focused specifically on graph modelling and visualization choices. The goal should be to decide which graph structures are scientifically honest, useful for exploration, and practical to compute from PostgreSQL-backed evidence statements.

The research phase should compare at least the following options:

- directed signed disease–taxon graphs versus typed-edge graphs such as `increased_in`, `decreased_in`, `no_change`, and `conflicting`;
- evidence-level multigraphs versus precomputed summary graphs;
- bipartite disease–taxon graphs versus projected disease–disease and taxon–taxon graphs;
- how to encode conflict visually without duplicating nodes;
- how to represent taxonomic rollup without hiding species-level contradictions;
- how source, method, sample type, geography, and review tier should affect edge weight;
- whether graph summaries should be stored as PostgreSQL materialized tables, cached JSON artifacts, or regenerated by Celery tasks;
- which renderer is best for each scale: Cytoscape.js for interactive networks, ECharts for matrix/heatmap-style summaries, and static exports for reports.

A good outcome of this research phase would be a short `docs/retes_design.md` document defining the official ORDINA graph semantics. It should specify the node types, edge types, edge attributes, aggregation rules, conflict metrics, projection rules, and visual encodings. This should be done before the graph layer becomes a flagship feature, because a visually attractive but semantically confused network would weaken the whole project.

11 Raw Data and Pipeline-derived Evidence

A future ORDINA pipeline remains important, but it should not be the first rewrite target. The pipeline should be introduced as one more source of evidence, parallel to DISBIOME and literature extraction.

A possible long-term flow is:

```
FASTQ / abundance table
-> QIIME 2 / Nextflow / taxonomic profiler
-> differential abundance or group comparison
```

-> ORDINA evidence contract
-> candidate evidence statement
-> review / provenance
-> graph update

This lets ORDINA combine:

- imported database evidence;
- manually reviewed literature evidence;
- semi-automated literature-mined candidates;
- reproducible pipeline-derived evidence.

This is a stronger long-term identity than a database alone.

12 Implementation Roadmap

The global project does not need to be rewritten, but the import subsystem should be replaced early. The current Django models, browser pages, taxonomy layer, and graph ideas are still useful. The part that should be scrapped is the old import implementation, because the new ORDINA direction depends on asynchronous source ingestion, candidate evidence generation, review queues, and graph recomputation.

12.1 Phase 1: Reposition and Freeze the Current Import Path

1. Update README and project documentation to position ORDINA as an evidence integration and network-analysis platform.
2. Mark the existing import implementation as legacy/prototype code.
3. Preserve the useful import concepts: contracts, preview, validation, provenance, and explicit batch records.
4. Avoid building new features on top of the old import path unless they are trivial and temporary.

12.2 Phase 2: Build the Worker-based ORDINA Flux Foundation

1. Add Redis to the Docker Compose stack.
2. Add one Celery worker container using the same Django codebase.
3. Add an `ImportRun` or `JobRun` model with status, progress, task identifier, timestamps, source, and error fields.
4. Add an API or admin action that creates a job row and enqueues a Celery task.
5. Keep PostgreSQL as the canonical state store and Redis as the broker/cache layer.

12.3 Phase 3: Source and Candidate Evidence Models

1. Add a `Source` model for external data sources.
2. Add `SourceDataset` and `SourceRecord` models for raw/staged source payloads.
3. Add `CandidateEvidenceStatement` and `ReviewDecision` models.
4. Add review states such as `imported_unreviewed`, `needs_review`, `approved`, `edited_and_approved`, and `rejected`.

12.4 Phase 4: DISBIOME Adapter as the First Real Worker Job

1. Build a DISBIOME import service that stores raw source records first.
2. Run the import as a Celery task triggered from Django.

3. Map DISBIOME records into candidate studies, groups, comparisons, taxa, and findings.
4. Mark imported records as `imported_unreviewed` unless manually checked.
5. Display DISBIOME provenance on evidence and graph pages.

12.5 Phase 5: Redesign Retes Semantics before Expanding the Graph UI

1. Treat the current graph views as prototypes, not as final graph semantics.
2. Add a `docs/retes_design.md` document defining node types, edge types, signs, aggregation rules, projection rules, and visual encodings.
3. Replace duplicated enriched/depleted organism nodes with canonical taxon nodes and directed, signed, evidence-backed edges.
4. Distinguish evidence-level multigraphs from aggregated summary graphs.
5. Move expensive graph summary generation into Celery tasks.
6. Add support/conflict summaries to disease-organism graph edges.
7. Add source, method, sample type, and review-tier filters to graph pages.
8. Add graph modes: all evidence, reviewed only, external only, manual only, conflicting only.
9. Add edge detail pages that show source breakdown, direction counts, review state, and the evidence statements behind each edge.

12.6 Phase 6: Curia Review Interface

1. Build a review queue for candidate evidence statements.
2. Add approve/edit/reject actions.
3. Store reviewer decisions and timestamps.
4. Keep rejected candidates for audit and model/rule evaluation.

12.7 Phase 7: Literature Mining

1. Add paper discovery from PubMed or Europe PMC queries.
2. Add candidate entity extraction using external annotations where possible.
3. Add rule-based direction extraction for simple cases.
4. Use reviewed decisions as training/evaluation data for future extraction models.

12.8 Phase 8: Pipeline-derived Evidence

1. Define an ORDINA abundance/differential-abundance import contract.
2. Build a small reproducible pipeline around QIIME 2, Nextflow, or a taxonomic profiler.
3. Run pipeline or conversion steps through dedicated worker tasks where appropriate.
4. Convert pipeline output into candidate evidence statements.
5. Keep pipeline evidence separate from literature evidence in provenance and graph filters.

13 What Not to Do

The shift should avoid three traps.

13.1 Do Not Become a DISBIOME Clone

ORDINA should not present itself as simply another manually curated association database. DISBIOME already does this well. ORDINA should treat DISBIOME-like data as input and focus on integration, provenance, review, and graph analysis.

13.2 Do Not Patch the Old Import System Indefinitely

The current import code should not become the foundation for DISBIOME synchronization, literature mining, graph precomputation, and pipeline-derived evidence. Patching it repeatedly would produce a fragile import layer. The safer move is to retire it as an implementation, preserve the useful ideas, and rebuild imports as job-based ORDINA Flux services.

13.3 Do Not Start with a Huge AI System

The manual curation problem is real, but a large NLP or LLM pipeline should not be the first deliverable. A better first step is to build the evidence and review infrastructure. Once candidate records, review states, and provenance are in place, NLP can be added incrementally.

13.4 Do Not Confuse Graph Visualization with Graph Semantics

The current graph layer should not simply be patched visually. The problem is not only appearance; it is graph meaning. Duplicating the same organism into enriched and depleted visual nodes is a semantic problem because it turns evidence state into entity identity. ORDINA should keep the useful prototype work, but redesign Retes around canonical nodes, directed signed edges, evidence summaries, and projection rules before treating the network view as a flagship feature. The renderer can evolve later; the graph model should be corrected first.

14 Conclusion

The discovery of DISBIOME should not stop ORDINA. It clarifies what ORDINA should be. The weak version of ORDINA is a second manually curated microbiome–disease database. The strong version is a provenance-first evidence integration system that turns external databases, literature-mined candidates, manual review, and pipeline-derived results into disease–organism networks.

The recommended strategic shift is therefore:

ORDINA should become the reviewable, source-aware, network-analysis layer for microbiome–disease evidence, with DISBIOME as an import source rather than a competitor to copy or replace.

This keeps the current Django-first architecture, preserves the useful parts of the existing schema and graph work, and gives the project a clearer niche: not collecting associations for their own sake, but making microbiome–disease evidence reproducible, traceable, conflict-aware, and analyzable as networks. The most important implementation correction is to rebuild the import layer early. ORDINA should keep PostgreSQL as the canonical database, but move long-running imports, source synchronization, candidate generation, and graph summaries into API-triggered Celery workers coordinated through Redis and deployed as Docker services. The most important network correction is to redesign Retes around canonical biological entities and evidence-backed relationships: one organism node should remain one organism node, while enrichment, depletion, uncertainty, and conflict should be represented on directed signed edges and their supporting evidence summaries.

References

- [1] Rafael Correia. *ORDINA repository README*. GitHub. Available at: <https://github.com/rafaelmdc/ORDINA>

- [2] Rafael Correia. *ORDINA schema documentation*. GitHub. Available at: <https://github.com/rafaelmdc/ORDINA/blob/main/docs/schema.md>
- [3] Rafael Correia. *ORDINA import pipeline documentation*. GitHub. Available at: https://github.com/rafaelmdc/ORDINA/blob/main/docs/import_pipeline.md
- [4] Rafael Correia. *ORDINA graph overview*. GitHub. Available at: <https://github.com/rafaelmdc/ORDINA/blob/main/docs/graph.md>
- [5] Rafael Correia. *ORDINA roadmap*. GitHub. Available at: <https://github.com/rafaelmdc/ORDINA/blob/main/docs/roadmap.md>
- [6] Rafael Correia. *ORDINA disease graph documentation*. GitHub. Available at: https://github.com/rafaelmdc/ORDINA/blob/main/docs/disease_graph.md
- [7] Rafael Correia. *ORDINA co-abundance graph documentation*. GitHub. Available at: https://github.com/rafaelmdc/ORDINA/blob/main/docs/co_abundance_graph.md
- [8] Y. Janssens et al. *Disbiome database: linking the microbiome to disease*. BMC Microbiology, 2018. DOI: <https://doi.org/10.1186/s12866-018-1197-5>
- [9] W. Hogan, A. Bartko, J. Shang, and C.-N. Hsu. *MiDRED: An Annotated Corpus for Microbiome Knowledge Base Construction*. Proceedings of the 23rd Workshop on Biomedical Natural Language Processing, 2024. DOI: <https://doi.org/10.18653/v1/2024.bionlp-1.31>
- [10] C.-H. Wei et al. *PubTator 3.0: an AI-powered literature resource for unlocking biomedical knowledge*. Nucleic Acids Research, 2024. DOI: <https://doi.org/10.1093/nar/gkae235>
- [11] Celery Project. *Introduction to Celery*. Available at: <https://docs.celeryq.dev/en/stable/getting-started/introduction.html>
- [12] Celery Project. *Using Redis*. Available at: <https://docs.celeryq.dev/en/stable/getting-started/backends-and-brokers/redis.html>